CALIBER: MAKING A GOVERNANCE CONTROL PLANE CHECK ITS OWN CLAIMS

Reza Rahimi [1]

ABSTRACT

Agentic systems fail across prompts, workflows, tools, MCP servers, and retrieval corpora whose release paths are usually fragmented. We describe CALIBER, a layered control plane that governs those heterogeneous resources through typed records, evidence-bearing release paths, and one operational surface spanning a browser UI, a served management API, a Python SDK, a CLI, and a plugin contract.

The engineering result we report is not the architecture alone. Building a system whose product is *governance* exposed a failure mode we did not anticipate and now consider the central lesson: a platform's stated guarantees drift from its implementation silently, because the artifacts that state them—documentation, capability metadata, dashboards, and gates—keep rendering correctly while becoming false. Over the development of CALIBER we recorded 22 audited findings and a recurring class of defect in which a control's description outran its behavior. A packaging gate asserting the shipped wheel contained a UI passed for twelve consecutive CI runs *by never executing*. A published-site gate reported success while the site returned 404. Two client test suites were counted as coverage while silently skipping. We responded by encoding platform claims as executable invariants—18 contract and ratchet suites that fail when a claim and its implementation disagree—and we report what they caught, including cases they caught in our own corrective work.

We state the evidence boundary once and plainly: CALIBER is open source, has been used at JazzX AI to build internal products with work under way to extend that usage, and is continuously exercised by 6,156 backend tests across 17 CI jobs—but it does not yet carry external production load, and we present no scale study. The contributions are the architecture, the invariant-checking methodology, and a measured taxonomy of how governance claims decay.

1 INTRODUCTION

Enterprise teams now operate agentic systems whose behavior depends on far more than a single prompt. A production incident may originate in a prompt, a workflow branch, an MCP server policy, a tool implementation, a retrieval corpus refresh, or an evaluation asset that drifted. Existing platforms solve part of this for prompts through versions, mutable aliases, and rollback, but the broader release surface of an agentic application remains spread across dashboards, source files, and runbooks (MLflow Project, 2026b; LangChain, Inc., 2026; Langfuse GmbH, 2026).

CALIBER addresses that gap as a control plane rather than as another composition framework. Agents and workflows may be authored elsewhere; CALIBER governs the resources they depend on and the path by which those resources change.

[1]JazzX AI. Correspondence to: Reza Rahimi <reza.rahimi@jazzx.ai>.

Proceedings of the 9^{th} MLSys Conference (Industry Track), Bellevue, WA, USA, 2026.

Building it surfaced a second problem that we believe generalizes beyond our system, and which forms the empirical core of this paper. When the product *is* governance, the platform's own claims become load-bearing artifacts: a capability flag that says a family supports rollback, a gate that says publication was verified, a document that says an endpoint accepts a field. Those artifacts share an unpleasant property. They fail *legibly*—the page still renders, the badge still shows green, the gate still reports success—so ordinary review does not catch them, and the failure is discovered by the operator who trusted them.

We found this class repeatedly and, eventually, systematically. Our response was to stop treating platform claims as prose and start treating them as assertions with executable counterparts. This paper reports the architecture, that methodology, and what it caught.

Contributions.

- A layered control-plane architecture for governed AI-agent resources in which adjacency confers *capability*

availability, not *capability inheritance*, and per-family release semantics are published rather than implied (§3).

- An intent-first release path that replaces unattainable cross-system atomicity with durable intent plus explicit reconciliation (§4).
- A methodology for making a governance platform's claims checkable, realized as 18 contract and ratchet suites over the running system, its generated artifacts, and its own CI configuration (§9).
- A measured taxonomy of governance-claim decay observed while building the system, including gates that pass by not running, generated artifacts that render correctly while being wrong, and dependency ranges that promise what no local run tests (§10).

Scope and honesty about evidence. We state this once, here, and do not repeat it as a hedge throughout. CALIBER is a substantial implemented system—327 Python modules, a React SPA, 490 route declarations across a served OpenAPI surface of 290 paths and 359 operations, 86 database migrations, and three separately published packages—under continuous integration with 6,156 backend tests at 93.2% statement coverage, 1,583 frontend tests, and 18 contract and ratchet suites, across 17 CI jobs. It is *not* a system with published production-scale operating data: we report no latency distribution under production load, no multi-tenant availability record, and no operator productivity study. Our own readiness assessment rates production readiness low and unchanged across four internal review cycles, because the operational blockers—externalized loop ownership, replica safety, and disaster-recovery numbers—are category gaps rather than quality gaps. Readers seeking a scale study will not find one here.

What the system does have is real internal use, a roadmap, and public availability. CALIBER is released as open source under Apache-2.0 and has been used at JazzX AI to build internal products—authoring and governing the agentic workflows those products depend on—rather than to serve external production traffic. Work is under way to extend that usage to further internal systems.

We are explicit about which kind of industrial paper this makes. It is a retrospective on a built and internally used system, and a report on a platform on a product roadmap; it is not a retrospective on a system carrying external production load, and we do not present it as one. The distinction matters for how the evidence should be read: the defects reported in §10 were found by people trying to build products with the platform, not by an evaluation designed to find them—which is what makes them worth reporting—but they are drawn from internal use rather than from a large operator population. Every measurement in this paper is reproducible from the public repository and its continuous-integration configuration.

2 Industrial Problem and Design Target

The core problem is not lack of observability. Systems can already trace requests, store prompts, and attach evaluation results (MLflow Project, 2026a). The missing piece is a *governed release path* across the non-code resources that actually determine agent behavior. Teams need to answer: Which workflow definition is live? Which knowledge-base build answered this incident? Which gate failed, and who overrode it? Can an operator roll back without redeploying the application?

These questions are harder than prompt versioning because the artifact families are genuinely heterogeneous. A prompt has a live alias. A test set is evidence and has no live target at all. A judge is evaluated for agreement but is not itself released. A workflow can publish an external service and restore a checkpoint. That heterogeneity drove the most consequential design decision in CALIBER, described next.

3 Architecture

CALIBER models each managed resource as a typed governed record with identity, version history, authority, and audit. Families span prompts, tools, skills, MCP servers, workflows, knowledge bases, test sets, judges, and agent anchors. Around that record shape, the platform organizes six lifecycle modes: author, test, evaluate, calibrate, release, and observe.

Around that record shape the platform organizes six lifecycle modes, and the separation matters operationally because each answers a different question and carries different authority. *Author* produces a new immutable version and never moves a live target. *Test* exercises a version in isolation and produces no evidence of record. *Evaluate* scores a version against a governed test set and produces evidence that binds to the version. *Calibrate* runs the improvement loop that proposes candidates from evidence. *Release* moves a live target and is the only mode that writes a release intent. *Observe* reads production behavior back. Conflating test with evaluate is the common failure in practice—it produces numbers that look like evidence but are not attached to anything releasable—so the modes are distinct in the API rather than in documentation.

The runtime is one Starlette ASGI application plus a React SPA served from the same origin. Operationally this matters because the browser UI, the management API, and published workflow services share one deployment boundary, one authentication model, and one audit substrate rather than

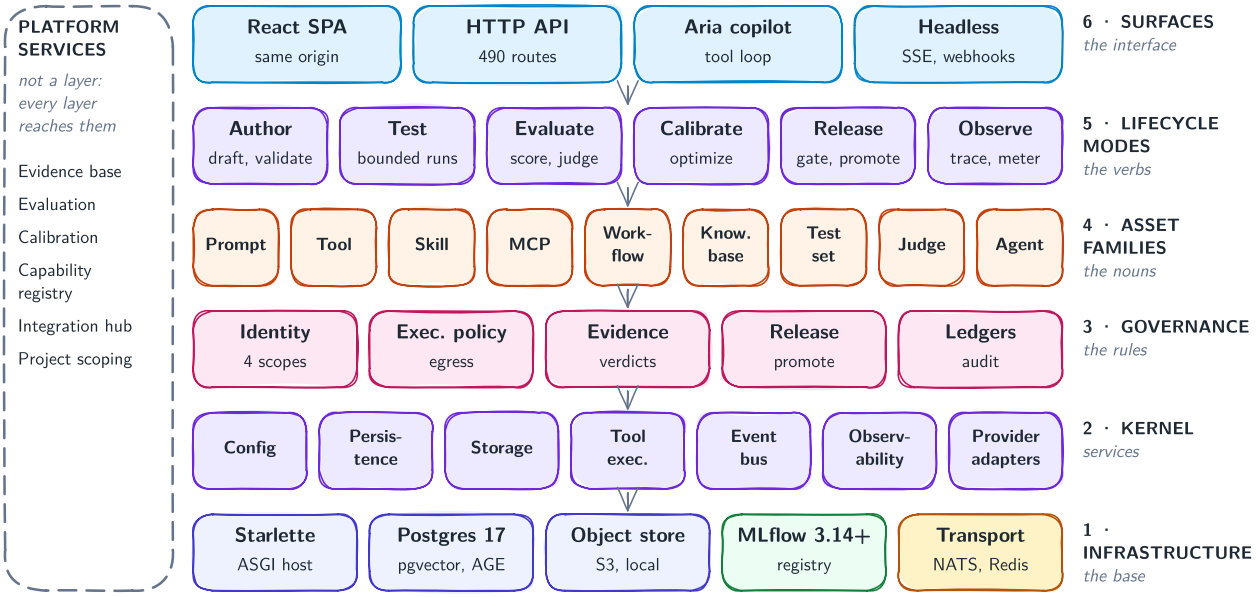

Figure 1. CALIBER is organized as a layered control plane. Shared substrate and governance capabilities sit beneath heterogeneous asset families and lifecycle modes; surfaces above them expose one operational system to browser users, automation, and extension authors.

competing side channels. The route table is mirrored into a served OpenAPI document (290 paths, 359 operations), which in turn backs a typed Python SDK and a CLI.

3.1 Availability is not inheritance

The architectural claim is deliberately narrow: adjacency in the layer diagram confers *capability availability*, not *capability inheritance*. A family placed in the governed-asset layer may use shared release, evidence, and audit primitives, but it acquires those guarantees only by explicitly wiring them.

This is not rhetorical caution. It is the only contract that stays true across heterogeneous families, and the alternative is actively dangerous. Four families in CALIBER report `rollbackable: true`, and the word means something different in each: alias restore for prompts, checkpoint-stack pop for workflow runs, derivation from activation history for knowledge bases, and writing a prior snapshot back as a *new* version for skills. An operator who sees one shared version-history component rendered in four places and infers one rollback guarantee has been misled by the interface, not by the documentation. CALIBER therefore publishes the per-family guarantee surface as machine-readable capability metadata that the UI must render, rather than as prose that the UI may contradict.

Governed family	Live target	Rollback mechanism
Prompt	alias	alias restore
Workflow	deployment	checkpoint-stack pop
Knowledge base	activation	derived from history
Skill	version	snapshot as new version
Tool, MCP server	registration	—
Test set, judge, anchor	none	—

Table 1. Nine governed families; four are rollbackable and no two mean the same thing by it. The capability record names the mechanism from a closed vocabulary, so a shared version-history component cannot imply a shared guarantee.

3.2 Capability metadata as a rendering obligation

Because the four rollback meanings above are genuinely different, CALIBER represents per-family guarantees as structured capability records rather than prose. Each family declares its history model, whether it has a live target, whether it is promotable, whether it is rollbackable, and—critically—*which* rollback mechanism applies, drawn from a closed vocabulary. A contract test requires the record to be complete for every family and requires any family claiming rollbackability to name a mechanism, so the ambiguous claim cannot be expressed.

The interface consumes that record instead of hardcoding assumptions. This is the concrete form of the paper's methodological argument: the difference between families is not

documented, it is *typed*, and a family added later cannot quietly inherit a guarantee it does not implement.

3.3 Broker-free execution

Background work is drained by 9 in-process loops using claim-and-lease ownership in the relational store rather than a dedicated broker. This keeps deployment lightweight and makes SQL the arbitration point, which suits an operator-facing control plane at moderate scale. The trade is explicit and is the system's principal scaling limitation: queue capacity and request capacity share one process model, and per-process limiters remain per-process until lifted into a shared store. Externalizing loop ownership is the gating prerequisite for multi-replica operation, and we have not built it.

4 Intent-First Release and Reconciliation

External registries and object stores do not join the caller's SQL transaction. Every governed prompt promotion therefore first writes a durable release intent recording the outgoing target, the incoming version, the acting principal, and the evidence reference; only then does it perform the external alias move. If the external system changes and local settlement does not, reconciliation inspects the durable intent and converges it.

This is not distributed atomicity. It is explicit, observable compensation in the saga tradition (Garcia-Molina & Salem, 1987), applied to a release path rather than to a long-running business transaction, and it inherits that tradition's premise—that systems which cannot hold a lock across participants must instead make their intent durable and their compensation explicit (Helland, 2007; Gray, 1981). The operationally valuable property is not an unattainable all-or-nothing guarantee but a durable record of intended change plus surfaces for reconciliation and abandonment. The same pattern recurs wherever the control plane must move state it does not own.

What differs from the classical formulation is the audit obligation. A saga needs compensation to restore consistency; a governance control plane additionally needs the intent to survive as evidence, because the question asked afterward is not only "is the system consistent" but "who changed this, against what evidence, and under whose authority." The intent record is therefore written before the external call *and* retained after it settles.

4.1 A governed promotion, end to end

To make the abstractions concrete, we trace one prompt promotion—the family with the strongest guarantee in the system—from authoring to rollback.

An author edits a prompt and registers a new immutable version; the live alias does not move. Evaluation runs against a versioned test set using a judge whose agreement with human labels has itself been scored, and the resulting scorecard is attached to the version as evidence rather than to a dashboard. Promotion is then requested. The release gate reads the attached evidence and either blocks or admits the change; a block is a state, not an error page, and it names the criterion that failed.

If the promotion is admitted, the system writes the durable release intent (§4) before touching the external registry: outgoing target, incoming version, acting principal, evidence reference. Only then does the alias move. The audit record is written from the intent, so the record of who promoted what against which evidence exists even if the external call fails afterward—and if it does fail, reconciliation has the intent it needs to converge or abandon explicitly.

Rollback restores the prior alias target. That is the prompt family's mechanism, and the capability record names it as such—so the interface showing rollback for a workflow beside rollback for a prompt is obliged to show that one pops a checkpoint stack while the other restores an alias.

Two properties of this path are worth naming because they were design decisions rather than conveniences. The evidence is bound to the version, not to a run, so a later reader can ask what justified a promotion without reconstructing a dashboard's state at that time. And the human decision is a state in the workflow rather than a side effect of a script continuing, which is what makes the audit record a decision record rather than a log line.

5 One Surface, Several Consumers

The management surface is shared by construction. The browser consumes the same governed-asset API shape the SDK does; the CLI sits on the SDK rather than on private admin calls; the OpenAPI document is generated from the live route table rather than hand-maintained, so a route that does not exist cannot be documented.

Three peer packages separate concerns that are genuinely different: `caliber-sdk` (a typed remote client, two runtime dependencies), `caliber-plugin-sdk` (a server-side extension contract with *zero* runtime dependencies), and `caliberctl` (non-interactive operator automation over the SDK). A developer script should not install MLflow, SQLAlchemy, and the server runtime merely to call a route; conversely a plugin contract should not depend on a live database or request context. Both properties are asserted against the built wheels in CI rather than trusted to review.

5.1 Extension as a supply-chain boundary

An optimizer plugin authors the artifact the platform promotes to production, which makes third-party registration a supply-chain surface rather than a convenience. Python entry points are a discovery mechanism, not an authorization one: any installed distribution, including one pulled in transitively, can advertise itself. CALIBER therefore discovers plugins automatically and enables none of them automatically—a plugin loads only when the deployment names its *distribution* in an allowlist, and an unlisted plugin is reported as installed and inert rather than hidden, so an operator can see what a wheel added.

This mirrors the direction of software supply-chain frameworks (Torres-Arias et al., 2019; Open Source Security Foundation, 2025): provenance is established by the consumer's policy rather than asserted by the artifact. The plugin's own claim about itself is never trusted—the registry records the distribution that metadata says shipped it, and marks every third-party optimizer experimental regardless of what the plugin declares.

A plugin also may not claim a name the platform ships. That reads as a naming convention and is not one: if a plugin could register the name of a built-in optimizer, installing a wheel would silently change what that name does for every agent already configured to use it—same name in every config, same name in every audit record, a different author's code producing the artifacts, and nothing in any diff.

6 Evidence, Judges, and Gates

A release gate is only as trustworthy as the evidence it reads, which makes the evaluation subsystem part of the governance story rather than an adjacent feature.

CALIBER treats evaluation assets as governed records in their own right. A test set is versioned and is evidence with no live target. A judge is a model-backed grader that is itself evaluated: the platform scores judge–human agreement on a labeled subset and reports it with chance-corrected statistics (Cohen, 1960; Fleiss, 1971), because a judge whose agreement is unmeasured turns a quality gate into an unverified opinion (Zheng et al., 2023; Liu et al., 2023). Retrieval corpora carry their own evaluation path (Es et al., 2024).

Two consequences shaped the design. First, evidence binds to the artifact version rather than to a dashboard state, so the justification for a past promotion is reconstructible without replaying a UI. Second, a gate verdict is a stored record with a criterion, a threshold, and an outcome—not a computed boolean—so a waiver is representable as a decision someone owns rather than as a threshold quietly lowered. Waivers are admin-only and audited, and signoff requires a rationale as a required field, since a signoff without a stated reason is not evidence that anyone decided anything.

The calibration loop closes this: refinement proposes a candidate, evaluation scores it against the versioned test set, and the gate admits or blocks. The human decision between candidate and promotion is a state in that loop rather than an implicit consequence of a script continuing.

7 Authorization and Fail-Closed Defaults

Three defaults in CALIBER are worth reporting because each was originally the convenient choice and each was changed after review.

Token scopes are a ceiling, not a grant. Automation credentials carry scopes that are intersected at every request with the owner's *current* scopes. Revoking a person's permission therefore shrinks their tokens' authority immediately, without enumerating tokens. The alternative—copying scopes at issuance—leaves a revoked administrator holding a still-administrative token, and that failure is invisible from the token list.

An unconfigured secret store refuses to serve. It returns an error naming the missing key source rather than degrading to unencrypted storage. The degradation path is the dangerous one precisely because it keeps working.

Host-path and tool-isolation defaults are closed. Both were originally open with documentation advising operators to close them. Documentation is not a control: the review that changed them noted that a single-layer defence at a policy boundary is exactly the pattern the review existed to find.

These are unremarkable individually. We report them because the pattern connecting them is the paper's thesis in a different register—each was a case where the system's stated posture and its default behavior had diverged, and the divergence was closed only when someone re-derived the mechanism rather than reading the description.

8 Deployment Topologies

CALIBER runs in two topologies that differ in infrastructure ownership rather than in application surface. The bundled topology packages PostgreSQL with pgvector and Apache AGE, object storage, MLflow, gateway support, and optional event backends behind loopback-only published ports, which makes a complete governed environment reproducible on one machine. The externalized topology swaps each of those for managed equivalents while preserving the same routes, the same authentication model, and the same audit substrate.

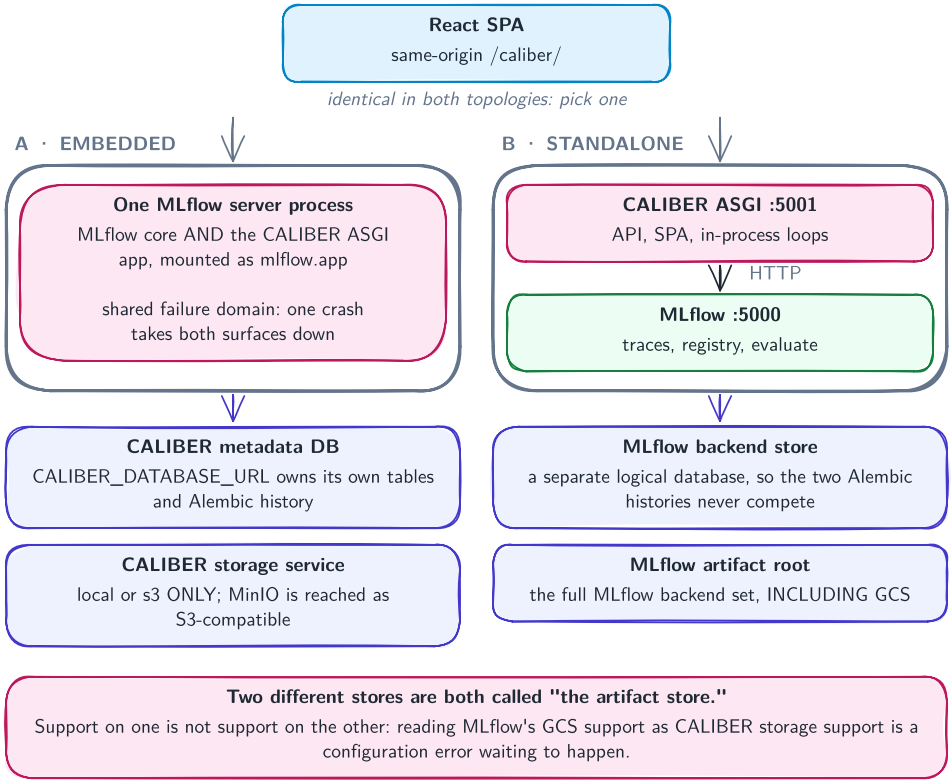

Figure 2. Two supported topologies: a loopback-oriented bundled stack for development, and an externalized deployment using managed infrastructure. The application surface is identical in both; only what sits beneath it changes.

Keeping the surface identical across both is what makes the bundled topology useful as more than a demo: the same contract suites run against both, so a behavior verified locally is a statement about the deployed system rather than about a simplified stand-in. The deployment contract itself is asserted—published ports must be loopback-only, and the container must retain its documented runtime hardening—because a topology diagram is a claim like any other.

9 Making Claims Checkable

The methodology this paper argues for is simple to state and was learned the hard way: in a governance platform, every claim the system makes about itself should have an executable counterpart that fails when the claim and the implementation disagree.

We realize this as 18 contract and ratchet suites. They differ from ordinary unit tests in what they take as the subject. A unit test asserts that a function computes; these assert that a *property of the system as a whole* still holds. Representative examples:

- **Capability contract.** Every artifact family must declare the complete capability record, and families reporting `rollbackable` must name a rollback *mechanism* from a closed vocabulary—so "rollback" cannot be asserted without saying which of the four distinct meanings applies.
- **Async-offload ratchet.** A counted bound on synchronous database calls inside async handlers, which may decrease but never increase.
- **Schema-coverage ratchet.** A bound on GA responses lacking a formal schema, moved from 52 to 8 with the count asserted exactly, so slack cannot silently reappear.
- **CI-local parity.** The local reproduction script must define a job for every CI job, so "it passes locally" remains a meaningful sentence.
- **Docs bijection.** Every published page must correspond to a source, and every source to a page.
- **Gate-execution ledger.** Each CI run emits which gates

actually executed, because a gate that never runs is indistinguishable from a gate that passes.

- **Dependency contract.** A declared version range must not admit a major version the code has not been ported to.

The ratchet form matters. A test that asserts an exact count rather than a bound with slack converts silent regression into a build failure, and—equally important—makes improvement visible as a deliberate edit. The schema-coverage baseline is accompanied by a test asserting the baseline is not slack, which caught our own off-by-one when the initial figure was transcribed from a grep that had counted an import line.

10 What the Invariants Caught

We group the recurring failures into four classes. All are drawn from the development record of this system; each is a case where ordinary review passed and an executable check did not.

10.1 Class 1: gates that pass by not running

The most instructive single defect was not a bug in the product. A packaging gate asserting that the shipped wheel contained the built SPA was correct, had existed from the beginning, and had been skipped on twelve consecutive CI runs because it sat behind six other jobs that were failing earlier. It executed for the first time only when a branch made every prerequisite pass—and failed immediately. The wheel had been shipping without a UI.

The generalizable form: *a gate that never executes is indistinguishable from a gate that passes*, and release checklists count both as coverage. A second instance followed a different route to the same place: a published-site gate reported success while the site served 404s, because it read the deployment workflow's exit code rather than the site. A third: two client test suites—the only ones driving the SDK and the plugin contract against the real application—were being cited as coverage while silently skipping in CI, because the job did not install the packages they import.

Our response was the gate-execution ledger: every run emits which gates ran, so absence becomes visible rather than inferred.

10.2 Class 2: artifacts that render correctly while being false

Generated artifacts fail with unusual quietness because their output looks plausible. Our published API reference, generated from source, shipped six defects of this kind simultaneously, each of which rendered perfectly:

Defect in generated API reference	Count
Cross-links emitted as literal text	82
Signatures naming an internal alias as a type	48
Request methods documenting no exceptions	19
Anchors pointing at nonexistent targets	3
Classes omitted from the reference entirely	3
A documented error envelope the server never sends	1

Table 2. Defects found in one generated documentation artifact. Every one rendered correctly in a browser; none was visible in the generator source.

Two are worth expanding. The cross-links were emitted inside code spans, and Markdown does not process links inside code spans—so 82 return types published their own link syntax as visible characters. The false error envelope documented fields the server has never sent; a reader following the reference exactly would have written code against a key that does not exist. A wrong example is worse than no example.

The methodological lesson is narrower and sharper than "test your docs." For generated artifacts, *the source is not the subject*. Every one of these defects is correct-looking generator code emitting wrong output, so the checks we added assert against generator *output*—including a sweep verifying that no published page links to an anchor it does not define.

A visual instance of the same class: a single CSS declaration (`overflow-wrap: anywhere` on table cells) reduced the min-content width used by automatic table layout, licensing the browser to shrink a column below its longest word. A six-column reference table whose true minimum width was 1034 px rendered at 778 px, and one label rendered in a 32 px column, 43 lines tall. The stylesheet was valid, the page rendered, and 2,997 measured occurrences were invisible to source review.

10.3 Class 3: promises about environments no local run tests

A declared dependency range is a promise about what a *clean* install may resolve, and no developer machine tests it—installed environments hold whatever they resolved months ago. We hit this four times. Most recently a range of `>=1.27,<3` admitted a 2.0 release on the day it shipped: CI installed it and reported five type errors, while every developer environment held the prior version and reported clean. The upper bound looked conservative and protected nothing.

The same asymmetry appears without dependencies. A test asserting a package's supported Python floor imported a module that is standard-library only *above* that floor, so the assertion about the floor could not run on it. And a

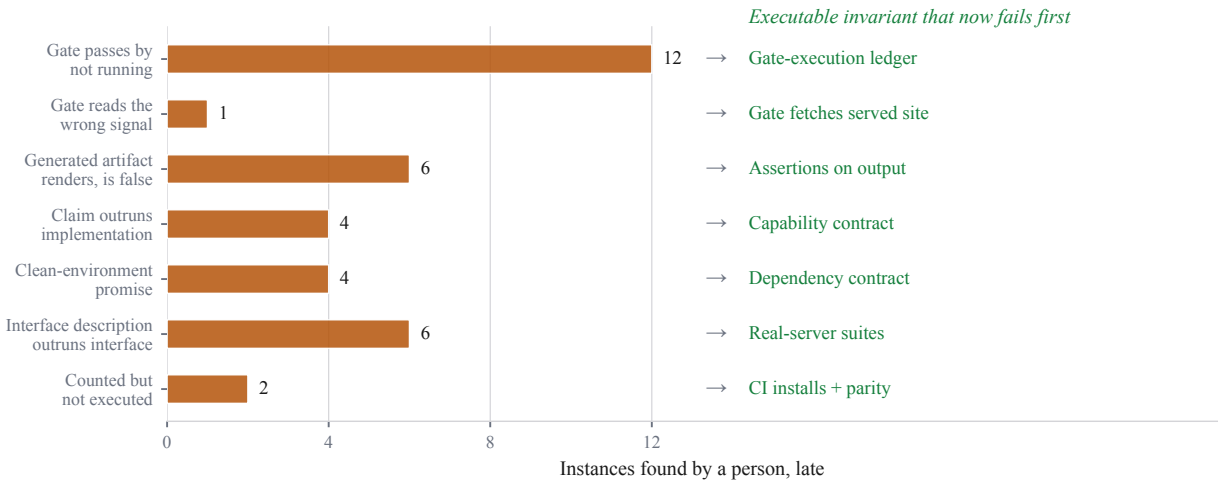

Figure 3. Every instance on the left was found by a person, after the fact. Every item on the right now fails in a build. Counts are instances recorded during development of CALIBER; the gate-execution class is dominated by a single gate that was skipped on twelve consecutive runs.

documentation test read a build artifact that is git-ignored, so it could never pass on a clean checkout—while a sibling test globbed the same missing directory and, because globbing a missing path yields nothing rather than raising, silently asserted against zero files while appearing to cover three copies.

10.4 Class 4: interface descriptions that outrun the interface

Client code written against a plausible reading of an API is confidently wrong in ways only the real server reveals. Driving our SDK against the running application in CI—rather than against mocks—found a listing endpoint that is POST-only (405), a resource surface that is workflow-scoped rather than global (404), a method naming a route that does not exist, a response schema silently dropping a field during serialization, and a schema that *invented* a null secret field on every row of a token listing. Mocked tests reproduced none of them, because mocks encode the same misreading as the client.

The cross-package instance is the sharpest. Our plugin SDK and our server deliberately do not import each other, which is the property that keeps plugins lightweight—and precisely the seam at which they diverged: the server's loader required its own internal type, so a plugin written against the plugin SDK's documented contract was rejected outright. Independence is a design goal and a drift risk simultaneously, and only a test that crosses the boundary observes both.

10.5 Closing the loop

Figure 3 pairs each class with the invariant that now catches it. We include it not to claim completeness—these checks bound known classes, and the next class will be discovered the way the others were—but because the pairing is the argument. Everything on the left was found by a person, late. Everything on the right fails in a build, before a human would look.

10.6 Audit findings and re-derivation

Separately, an external correctness and security review produced 22 findings, which we re-derived from source rather than accepting. Nineteen were confirmed, six partly, and one refuted. The result we consider transferable is not the ratio but this: the severity assessments were largely sound while the stated *mechanisms* were wrong often enough that acting on the ticket text without re-derivation would have produced the wrong fix in at least three cases. An audit finding is a hypothesis about a system, not an observation of it.

11 LESSONS

Cross-family release semantics are the real problem. Prompt lifecycle management is no longer novel by itself. The engineering problem is extending governed release discipline to workflows, tools, MCP servers, knowledge versions, and review assets *without* pretending they behave like prompts.

Shared UI is not shared semantics. The most likely operator error in our system is inferring one rollback guarantee

from one rendered component appearing in four places. A platform that reuses interface elements across families must publish the differences in a form the interface is obliged to render.

Durable intent beats unattainable atomicity. When the control plane moves state it does not own, the useful guarantee is a durable record of intended change plus explicit reconciliation—not a transaction that cannot exist.

Work that stops for a person is neither running nor finished. A governed system pauses for human decisions, and modeling that as a transient state is a category error with operational consequences. A refinement job that produced a candidate has stopped and will never advance on its own. Treating it as running makes a client wait out its timeout; treating it as failed reports a broken pipeline for a system working correctly. We model it as a first-class state, and our CLI gives it a distinct exit code, because pipelines must branch on it.

Discovery is not authorization. For any extension point whose code gains authority over production artifacts, the safe default is that installation makes a plugin visible and an explicit deployment decision makes it active.

A per-file test run is a weaker question than the full run. Changing a *default* is never local to the code that reads it: every test that never mentioned the setting was relying on it. We recorded this three times, once after a focused slice reported green and the full suite reported 66 failures in files whose names suggested neither subject.

Encode the invariant when you fix the instance. Every failure class above recurred before it was pinned. The marginal cost of adding the executable check while the cause is understood is far below the cost of rediscovering it, and the check preserves the reasoning—which prose in a commit message does not.

11.1 What we would do differently

Three pieces of guidance are transferable to teams building platforms whose product is an assurance rather than a computation.

Write the invariant before the documentation. Most of our claim-decay defects entered when a document or capability flag was authored ahead of, or alongside, the mechanism it described. The document then became the specification nobody executed. Inverting the order costs little at the time and removes the class.

Make gate execution a first-class signal. We spent twelve CI runs believing a packaging gate protected us. Any release process with conditional or dependent gates should emit which gates ran, not only which passed; "did not run" and "passed" must not be the same colour on a dashboard.

Verify against the running system at the boundary you claim to support. Our mocked client tests encoded the same misreading as the client and reproduced none of the six interface defects that driving against the real application found. For any published interface, at least one suite should cross the real boundary—and it must be installed in CI, because a suite that skips is counted as coverage while providing none.

We would also start the operational work earlier. Our readiness assessment moved on security and release engineering across four review cycles and did not move at all on production readiness, because every cycle improved what existed rather than building the externalized loop ownership that multi-replica operation requires. That is a defensible sequencing choice for a system seeking design confidence first, but it should be made deliberately rather than discovered in a table that refuses to move.

12 Related Work

Prompt and asset registries. Prompt registries provide versions, mutable aliases, and rollback for a single family (MLflow Project, 2026b; LangChain, Inc., 2026; Langfuse GmbH, 2026), and tracing and evaluation stacks supply observability and scoring (MLflow Project, 2026a; Arize AI, 2025; promptfoo, 2025; OpenAI, 2025b). CALIBER consumes rather than replaces these: it reuses MLflow (Zaharia et al., 2018) as substrate for prompt registry, tracing, and gateway concerns. The gap it targets is that these systems govern one family well and leave the cross-family release path—workflows, tools, MCP servers (Anthropic, 2024), knowledge versions, evaluation assets—to dashboards and runbooks.

Pipeline and lifecycle platforms. TFX (Baylor et al., 2017), Kubeflow (Kubeflow Authors, 2025), Airflow (Apache Software Foundation, 2025), and DVC (Iterative, Inc., 2025) govern the lifecycle of *training and data* artifacts, with provenance as a first-class concern (Schelter et al., 2017). CALIBER addresses the analogous problem one layer up, where the governed artifacts are the non-code resources that determine an agent's behavior at inference time, and where the release act is an alias move or a service publication rather than a pipeline run.

Documentation as governance artifact. Model cards (Mitchell et al., 2019) and datasheets (Gebru et al., 2021) established that a system's claims about itself are part of its interface. Our experience sharpens that in one direction: such artifacts decay silently, and the decay is invisible to review because they continue to render. The

invariants in §9 are an attempt to give those claims a failing build rather than a stale page.

Testing and technical debt in ML systems. Our methodology is closest in spirit to the ML Test Score (Breck et al., 2017) and to the hidden-technical-debt line of work (Sculley et al., 2015; Amershi et al., 2019), which argue that ML systems accumulate risk in the infrastructure around the model rather than in the model. We extend the framing to a governance platform, where the accumulated risk is specifically that the system's assurances outlive their implementations. The ratchet form—an exact bound that may improve but never regress—is a fitness function in the continuous-delivery sense (Humble & Farley, 2010; Beyer et al., 2016).

Transactions and compensation. The intent-first release path is a saga with an audit obligation (Garcia-Molina & Salem, 1987; Gray, 1981), and its premise is Helland's: systems spanning trust or storage boundaries cannot hold a lock across them and must make intent durable instead (Helland, 2007; Kleppmann, 2017).

Supply chain and policy. The plugin allowlist follows the direction of in-toto (Torres-Arias et al., 2019) and SLSA (Open Source Security Foundation, 2025): the consumer's policy establishes provenance rather than the artifact's own assertion. Policy engines such as OPA (Cloud Native Computing Foundation, 2025) externalize authorization decisions; CALIBER keeps its authorization in-process but makes the resulting posture machine-readable for the same reason.

Agent frameworks. Composition frameworks (LangChain, Inc., 2025; LlamaIndex, Inc., 2025; CrewAI, Inc., 2025; Wu et al., 2024; OpenAI, 2025a) and the optimizer literature (Khattab et al., 2024; Opsahl-Ong et al., 2024; Agrawal et al., 2025) address orchestration and prompt improvement respectively. CALIBER is deliberately not a competing composition framework: it governs the resources those frameworks consume, and it treats optimizers as a registered, allowlisted extension point rather than as a built-in capability.

13 LIMITATIONS

The evidence boundary stated in §1 is the principal limitation and we do not restate it as a hedge: there is no production-scale operating data here.

Beyond that, three limits are structural. Guarantees remain per family: the platform publishes a checked capability surface but does not yet prove that every declared capability is implemented by every handler claiming it. Execution remains single-process: multi-replica operation is gated on externalizing loop ownership, which is unbuilt. Autonomy remains scoped: assistant-driven draft review exists, but it is not evidence that open-ended autonomous release is safe, and the enforced human decision in the refinement path is a deliberate boundary rather than an unfinished feature.

Finally, the taxonomy in §10 is drawn from one system built by a small team. We report frequencies within that record, not incidence rates for the industry, and the classes should be read as hypotheses that generalize rather than as measured population statistics.

14 CONCLUSION

CALIBER is an implemented answer to a practical enterprise problem: governing the non-code resources that determine how agentic systems behave. Its architectural result is that heterogeneous resources can share one control plane while keeping release guarantees explicitly family-specific—a narrower claim than a universal release contract, and the one the system actually supports.

It is also a system on a roadmap rather than a finished one, and the two statements are related: the invariants described here exist because extending a governance platform to more internal products is precisely the moment its stated guarantees start being relied on by people who did not write them.

Its engineering result is the one we did not expect to be writing. A platform whose product is governance must treat its own claims as load-bearing artifacts, because those claims fail legibly: the page renders, the badge is green, the gate reports success, and the operator who trusted them discovers otherwise. Encoding claims as executable invariants converted that class of failure from something we discovered in production-adjacent surprise into something a build fails on. We would make that investment earlier next time, and we would make it before writing the documentation that describes what the system guarantees.

AVAILABILITY

CALIBER is open source under Apache-2.0. The repository, including the continuous-integration configuration that produces every figure and count in this paper, is at `https://github.com/rrahimi-uci/caliber-suite`.

REFERENCES

Agrawal, L. A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Stoica, I., Khattab, O., and Zaharia, M. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint*

arXiv:2507.19457, 2025.

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.

Anthropic. Model context protocol specification. https://modelcontextprotocol.io, 2024. Accessed 2026-08-03.

Apache Software Foundation. Apache Airflow. https://airflow.apache.org, 2025. Accessed 2026-08-03.

Arize AI. Phoenix: AI observability and evaluation. https://github.com/Arize-ai/phoenix, 2025. Accessed 2026-08-03.

Baylor, D., Breck, E., Cheng, H.-T., Fiedel, N., Foo, C. Y., Haque, Z., Haykal, S., Ispir, M., Jain, V., Koc, L., Koo, C. Y., Lew, L., Mewald, C., Modi, A. N., Polyzotis, N., Ramesh, S., Roy, S., Whang, S. E., Wicke, M., Wilkiewicz, J., Zhang, X., and Zinkevich, M. TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2017.

Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data (BigData)*, 2017.

Cloud Native Computing Foundation. Open policy agent. https://www.openpolicyagent.org, 2025. Accessed 2026-08-03.

Cohen, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 1960.

CrewAI, Inc. CrewAI: Framework for orchestrating role-playing autonomous agents. https://github.com/crewAIInc/crewAI, 2025. Accessed 2026-08-03.

Es, S., James, J., Espinosa Anke, L., and Schockaert, S. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024.

Fleiss, J. L. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 1971.

Garcia-Molina, H. and Salem, K. Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1987.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., and Crawford, K. Datasheets for datasets. *Communications of the ACM*, 64(12), 2021.

Gray, J. The transaction concept: Virtues and limitations. *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, 1981.

Helland, P. Life beyond distributed transactions: An apostate's opinion. In *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007.

Humble, J. and Farley, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

Iterative, Inc. DVC: Data version control. https://dvc.org, 2025. Accessed 2026-08-03.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

Kubeflow Authors. Kubeflow pipelines. https://www.kubeflow.org/docs/components/pipelines/, 2025. Accessed 2026-08-03.

LangChain, Inc. LangChain and LangGraph. https://github.com/langchain-ai/langchain, 2025. Accessed 2026-08-03.

LangChain, Inc. Manage prompts in LangSmith. https://docs.langchain.com/langsmith/manage-prompts, 2026. Documents commit-based version history, custom commit tags, the reserved `staging` and `production` tags, `client.pull_prompt("name:production")` resolution, rollback to a previous commit, and owners-only tag permissions. Accessed 2026-08-03.

Langfuse GmbH. Prompt version control. https://langfuse.com/docs/prompt-management/features/prompt-version-control, 2026. Documents version identifiers, the `production` and `latest` labels plus custom labels, client-side resolution by label at fetch time, rollback by re-assigning the `production` label, and protected labels restricted by role. Accessed 2026-08-03.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

LlamaIndex, Inc. LlamaIndex: Data framework for LLM applications. https://github.com/run-llama/llama_index, 2025. Accessed 2026-08-03.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019.

MLflow Project. MLflow for GenAI: Tracing, evaluation, and assessments. https://mlflow.org/docs/latest/genai/, 2026a. The current-capability source for claims about MLflow traces, assessments, scorers, and evaluation entry points. The 2018 paper predates all of these and is cited only for MLflow's original contribution. Accessed 2026-08-03.

MLflow Project. MLflow prompt registry. https://mlflow.org/docs/latest/genai/prompt-registry/, 2026b. Documents commit-based prompt versioning, mutable aliases, prompt- and version-level tags, commit messages, and immutable versions. Accessed 2026-08-03.

Open Source Security Foundation. SLSA: Supply-chain levels for software artifacts. https://slsa.dev, 2025. Accessed 2026-08-03.

OpenAI. OpenAI agents SDK. https://github.com/openai/openai-agents-python, 2025a. Accessed 2026-08-03.

OpenAI. OpenAI evals. https://github.com/openai/evals, 2025b. Accessed 2026-08-03.

Opsahl-Ong, K., Ryan, M. J., Purtell, J., Broman, D., Potts, C., Zaharia, M., and Khattab, O. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

promptfoo. promptfoo: Test and evaluate LLM prompts. https://github.com/promptfoo/promptfoo, 2025. Accessed 2026-08-03.

Schelter, S., Boese, J.-H., Kirschnick, J., Klein, T., and Seufert, S. Automatically tracking metadata and provenance of machine learning experiments. In *Machine Learning Systems Workshop at NeurIPS*, 2017.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. in-toto: Providing farm-to-table guarantees for bytes and bits. In *28th USENIX Security Symposium*, 2019.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., and Zumar, C. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 2018.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.